WEEK 3 · PHASE 2

Data Ingestion & Loading

COPY INTO · Incremental Load · File Formats · JDBC/ODBC · Schema Evolution

- COPY INTO command syntax & options
- Incremental loading from ADLS/S3/GCS
- Idempotent ingestion patterns
- File formats: CSV, JSON, Parquet, Avro, ORC
- JDBC/ODBC/REST client integration
- Schema enforcement & evolution
- Practice Questions (15 Qs) + Answer Key

Databricks Certified Data Engineer Associate | Study Guide Week 3 | Generated August 2026

Table of Contents

1. COPY INTO Command..... 3
2. Incremental Loading Patterns..... 5
3. File Formats Deep Dive..... 8
4. Idempotent Ingestion Design..... 11
5. Cloud Storage Integration..... 13
6. JDBC/ODBC/REST Clients (NEW)..... 15
7. Schema Enforcement & Evolution (NEW)..... 18
8. Practice Questions..... 21
9. Answer Key..... 25
10. Self-Assessment Checklist..... 27
BONUS: Advanced Patterns & Lab..... 28
BONUS: Week 3 Cheat Sheet..... 32

SECTION 1

1. COPY INTO Command

Batch ingestion into Delta tables

COPY INTO is a SQL command that loads data from cloud storage into a Delta Lake table. It is idempotent by default — running it multiple times will not load the same files twice, making it safe for scheduled batch ingestion pipelines.

1.1 Basic Syntax

■ COPY INTO — Basic Syntax

```
COPY INTO target_table
FROM 'abfss://container@account.dfs.core.windows.net/path/'
FILEFORMAT = PARQUET
FORMAT_OPTIONS ('mergeSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
```

1.2 Key Parameters

Parameter	Description	Example Value
FILEFORMAT	Source file format	CSV, JSON, PARQUET, AVRO, ORC, TEXT
FORMAT_OPTIONS	Format-specific settings	"header"="true", "delimiter"=","
COPY_OPTIONS	Behavior options	"mergeSchema"="true", "force"="false"
FILES	Specific file list (optional)	FILES = ('file1.csv', 'file2.csv')
PATTERN	Glob pattern filter	PATTERN = '*.parquet'
VALIDATE	Dry-run validation mode	VALIDATE ALL / VALIDATE N ROWS

■ COPY INTO tracks loaded files in Delta transaction log. Re-running will skip already-loaded files unless COPY_OPTIONS ('force' = 'true') is set.

1.3 CSV-Specific Options

■ COPY INTO — CSV with all options

```
COPY INTO sales_bronze
FROM 's3://mybucket/raw/sales/'
FILEFORMAT = CSV
FORMAT_OPTIONS (
 'header' = 'true',
 'delimiter' = ',',
 'quote' = '"',
 'escape' = '\\',
 'nullValue' = 'NULL',
 'inferSchema' = 'true'
);
```

1.4 VALIDATE Mode

Use VALIDATE to perform a dry-run without writing data. This checks for schema mismatches, format errors, and permission issues before the actual load.

■ COPY INTO — VALIDATE mode

```
-- Validate all files
COPY INTO target_table
FROM 's3://bucket/path/'
FILEFORMAT = JSON
VALIDATE ALL;

-- Validate only first 100 rows
COPY INTO target_table FROM 's3://bucket/path/'
FILEFORMAT = JSON VALIDATE 100 ROWS;
```

SECTION 2

2. Incremental Loading Patterns

Batch vs streaming design

Incremental loading processes only new or changed data, reducing compute costs and latency compared to full table reloads. Databricks supports several patterns depending on data arrival frequency.

2.1 Ingestion Method Comparison

Feature	COPY INTO	Auto Loader	Lakeflow Connect
Interface	SQL command	PySpark (cloudFiles)	Managed connectors
Trigger	Manual / scheduled	Streaming (file notification)	Managed scheduler
Schema Inference	Single pass	Incremental & evolving	Target-driven
File Scale	Up to millions	Billions (file notification)	Depends on connector
Use Case	Batch loads, migrations	Streaming event streams	Enterprise SaaS/Databases
State Tracking	Delta transaction log	Checkpoint directory	Connector state

■ **COPY INTO always uses directory listing mode. Auto Loader can switch between directory listing (all files) and file notification mode (delta events only).**

2.2 Idempotent Pattern with COPY INTO

Because COPY INTO tracks which files have been loaded via Delta's transaction log, it is inherently idempotent. Schedule it to run periodically — only new files in the source path will be ingested.

■ **Incremental COPY INTO — Hourly Batch**

```
-- Scheduled job runs this every hour
-- Only NEW files since last run are loaded
COPY INTO events_bronze
FROM 'abfss://raw@storage.dfs.core.windows.net/events/'
FILEFORMAT = JSON
FORMAT_OPTIONS ('multiLine' = 'false');

-- Check what was loaded
DESCRIBE HISTORY events_bronze;
```

2.3 Partitioned Source Paths

Loading from date-partitioned paths reduces the scan range and improves performance. Use parameterized paths in Databricks Jobs to pass the current date.

■ **Partitioned path ingestion**

```
-- Load only today's partition
DECLARE OR REPLACE VARIABLE load_date DATE = CURRENT_DATE;
COPY INTO orders_bronze
FROM CONCAT(
 's3://datalake/orders/date=',
 CAST(load_date AS STRING)
)
FILEFORMAT = PARQUET;
```

■ **When using partitioned paths with COPY INTO, use PATTERN = '*.parquet' to avoid scanning non-data files like _SUCCESS or _metadata.**

SECTION 3

3. File Formats Deep Dive

Parquet, Avro, ORC, JSON, CSV

Choosing the right file format impacts read performance, compression, schema evolution support, and compatibility with downstream systems.

3.1 Format Comparison Matrix

Format	Layout	Schema	Compression	Best For
Parquet	Columnar	Embedded	Snappy/ZSTD	Analytics, large tables
Delta	Columnar + log	Embedded	Snappy/ZSTD	Lakehouse (ACID, time travel)
ORC	Columnar	Embedded	ZLIB/Snappy	Hive-compatible workloads
Avro	Row-based	JSON schema	Snappy/Deflate	Streaming, Kafka events
JSON	Row-based	Self-describing	None (gzip opt)	APIs, nested/semi-structured
CSV	Row-based	None	None (gzip opt)	Simple exports, Excel compat

3.2 Parquet — The Lakehouse Standard

Parquet is the default format for Delta Lake tables. Its columnar layout allows Spark to read only the columns needed for a query (column pruning) and apply min/max statistics for data skipping, dramatically improving query performance.

■ Parquet write options

```
-- Write as Parquet (default for Delta)
df.write.format("delta").save("/mnt/silver/sales")

-- Explicit Parquet write with partitioning
df.write.format("parquet")
 .option("compression", "snappy")
 .partitionBy("year", "month")
 .save("/mnt/raw/events")
```

3.3 Avro — Streaming & Kafka

Avro is row-based and stores its schema within the file (JSON format), making it ideal for Kafka-based streaming pipelines where schema evolution is common. Databricks supports reading Avro with cloudFiles and COPY INTO.

■ Avro ingestion

```
COPY INTO events_bronze
FROM 's3://kafka-sink/events/'
FILEFORMAT = AVRO;

-- PySpark read
df = spark.read.format("avro").load("s3://kafka-sink/events/")
```

3.4 JSON — Nested & Semi-Structured

■ JSON ingestion & nested access

```
-- Multi-line JSON (one object spans multiple lines)
COPY INTO raw_json_table
FROM 's3://bucket/json-files/'
```

```
FILEFORMAT = JSON
FORMAT_OPTIONS ('multiLine' = 'true');

-- Parse nested fields after ingestion
SELECT
 raw:user.id AS user_id,
 raw:user.address.city AS city,
 raw:events[0].type AS first_event
FROM raw_json_table;
```

■ **For nested JSON, ingest as STRING first (raw column), then parse with get_json_object(), from_json(), or : (colon) operator in Databricks SQL.**

SECTION 4

4. Idempotent Ingestion Design

Fault-tolerant patterns

Idempotency means running the same ingestion operation multiple times produces the same result. This is critical for fault-tolerant pipelines that may retry on failure.

4.1 Why Idempotency Matters

Scenario	Without Idempotency	With Idempotency (COPY INTO)
Job retry after failure	Duplicate rows in table	No duplicates — already loaded files skipped
Scheduled overlap	Double-count in reports	Safe — state tracked in transaction log
Reprocessing	Manual dedup required	force=true reloads cleanly
Multiple job runs	Exponential data growth	Linear growth — one load per unique file

4.2 Delta Lake + COPY INTO = Idempotent

■ **Idempotent COPY INTO behavior**

```
-- Delta transaction log records each loaded file:
-- {'path': 's3://bucket/data/file1.parquet', 'loaded': true}

-- COPY INTO checks log before loading:
-- IF file already in log → SKIP
-- IF file not in log → LOAD and record

-- Result: run daily, only new files load each day
COPY INTO bronze_table FROM 's3://bucket/incoming/'
FILEFORMAT = PARQUET; -- Safe to re-run!
```

4.3 MERGE-based Upsert Pattern

For SCD (slowly changing dimensions) or deduplication on top of COPY INTO, use MERGE INTO after loading to bronze to promote cleansed data to silver:

■ **Bronze → Silver MERGE pattern**

```
-- Step 1: Load to bronze (idempotent)
COPY INTO orders_bronze FROM 's3://bucket/orders/'
FILEFORMAT = PARQUET;

-- Step 2: Upsert from bronze to silver
MERGE INTO orders_silver AS target
USING orders_bronze AS source
ON target.order_id = source.order_id
WHEN MATCHED AND source.updated_at > target.updated_at
 THEN UPDATE SET *
WHEN NOT MATCHED
 THEN INSERT *;
```

SECTION 5

5. Cloud Storage Integration

S3, ADLS, GCS & External Locations

Databricks integrates with AWS S3, Azure ADLS Gen2, and Google Cloud Storage. Access is configured via Unity Catalog external locations or cluster-level credentials.

5.1 Storage Path Formats

Cloud Provider	Path Format	Example
AWS S3	s3://bucket/path/	s3://my-lake/raw/events/
Azure ADLS Gen2	abfss://container@account.dfs.core.windows.net/	abfss://bronze@acct.dfs.core.windows.net/sales/
Google Cloud Storage	gs://bucket/path/	gs://my-gcs-bucket/data/
DBFS (Legacy)	dbfs:/... (avoid)	dbfs:/mnt/data/ (avoid for new work)

5.2 External Locations (Unity Catalog)

External Locations in Unity Catalog replace legacy DBFS mount points. They provide governed, credential-free access to cloud storage.

■ **Unity Catalog — External Location setup**

```
-- Create credential (admin)
CREATE STORAGE CREDENTIAL my_cred
 WITH (AZURE MANAGED IDENTITY);

-- Create external location
CREATE EXTERNAL LOCATION raw_zone
 URL 'abfss://raw@storage.dfs.core.windows.net/'
 WITH (STORAGE CREDENTIAL my_cred);

-- Grant access to data engineers
GRANT READ FILES ON EXTERNAL LOCATION raw_zone TO `data_engineers`;
```

■ **External Locations replace legacy mount points (dbutils.fs.mount). Unity Catalog controls access at the storage level — no secrets needed in notebooks.**

SECTION 6

6. JDBC/ODBC/REST Clients

Direct database integration

Beyond cloud storage, Databricks supports direct ingestion from databases via JDBC/ODBC or REST APIs. These patterns are commonly orchestrated with Lakeflow Jobs.

6.1 JDBC Connection Pattern

JDBC (Java Database Connectivity) allows Spark to read directly from databases like PostgreSQL, MySQL, Oracle, SQL Server, and others. The connection is pulled into Spark as a DataFrame and can be written to Delta tables.

■ JDBC — Direct database read

```
# PySpark JDBC read
df = spark.read \
 .format("jdbc") \
 .option("url", "jdbc:postgresql://db.example.com:5432/mydb") \
 .option("dbtable", "public.customers") \
 .option("user", dbutils.secrets.get(scope="dbfs", key="db_user")) \
 .option("password", dbutils.secrets.get(scope="dbfs", key="db_pass")) \
 .load()

# Write to Delta bronze table
df.write.format("delta").mode("overwrite") \
 .option("mergeSchema", "true") \
 .save("/mnt/bronze/customers")
```

6.2 Incremental JDBC with Cursor Columns

Use a cursor column (timestamp, ID, or sequence number) to track which rows have been loaded, enabling incremental ingestion without CDC (Change Data Capture):

■ Incremental JDBC — Cursor-based

```
# Read only new rows since last load
last_load_ts = spark.read.parquet("/mnt/bronze/customers") \
 .agg({"updated_at": "max"}) \
 .collect()[0][0]

df = spark.read \
 .format("jdbc") \
 .option("url", "jdbc:postgresql://db.example.com:5432/mydb") \
 .option("dbtable", "(SELECT * FROM customers WHERE updated_at > '"+ str(last_load_ts) +"') AS q") \
 .load()

df.write.format("delta").mode("append").save("/mnt/bronze/customers")
```

6.3 REST API Ingestion

For APIs without JDBC drivers (e.g., cloud SaaS, internal microservices), use Python requests to fetch data and convert to DataFrame:

■ REST API — Direct ingestion

```
import requests, json
from pyspark.sql import Row
```

```
# Fetch from REST API
url = "https://api.example.com/v1/events"
headers = {"Authorization": f"Bearer {dbutils.secrets.get('api', 'token')}"}
response = requests.get(url, headers=headers)
data = response.json()["results"]

# Convert to DataFrame
df = spark.createDataFrame([Row(**row) for row in data])
df.write.format("delta").mode("append").save("/mnt/bronze/events")
```

■ **For enterprise connectors (Salesforce, ServiceNow, Oracle), use Lakeflow Connect managed connectors instead of custom JDBC/REST code. Lakeflow Connect handles CDC, schema evolution, and credential management.**

SECTION 7

7. Schema Enforcement & Evolution

Handling schema changes

As data sources evolve, schemas change. Databricks provides two strategies to handle this: schema enforcement (strict, fail on mismatch) and schema evolution (accept and merge).

7.1 Schema Enforcement (Strict)

Schema enforcement rejects data that doesn't match the target table schema. This catches data quality issues early but requires proactive schema management.

■ Schema Enforcement — Strict (default)

```
-- Schema enforcement ON (default)
COPY INTO orders_bronze
FROM 's3://bucket/orders/'
FILEFORMAT = PARQUET;
-- Result: FAIL if source has extra/missing columns
```

7.2 Schema Evolution (Flexible)

Schema evolution allows new columns from the source to be added to the target table. This is critical when data sources evolve (e.g., new fields in JSON API responses):

■ Schema Evolution — Flexible

```
-- Schema evolution ON
COPY INTO orders_bronze
FROM 's3://bucket/orders/'
FILEFORMAT = PARQUET
FORMAT_OPTIONS ('mergeSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
-- Result: New columns added to target table
-- Existing rows get NULL for new columns
```

7.3 Auto Loader Schema Handling (Preview)

Auto Loader (cloudFiles) supports incremental schema inference. On each batch, it can detect new columns and add them to the checkpoint schema. This is covered in detail in Week 4.

■ Auto Loader — Incremental schema inference

```
# Auto Loader with schema evolution
df = spark.readStream \
 .format("cloudFiles") \
 .option("cloudFiles.format", "parquet") \
 .option("cloudFiles.schemaLocation", "/mnt/checkpoint/.schema") \
 .option("cloudFiles.schemaEvolutionMode", "addNewColumns") \
 .load("s3://bucket/incoming/")

df.writeStream \
 .format("delta") \
 .option("checkpointLocation", "/mnt/checkpoint/orders") \
 .option("mergeSchema", "true") \
 .mode("append") \
 .start("/mnt/bronze/orders")
```

■■ **Schema evolution with mergeSchema requires the option in BOTH FORMAT_OPTIONS AND COPY_OPTIONS. Missing either setting will cause the command to fail.**

Practice Questions (Week 3)

1. A data engineer runs COPY INTO on a Delta table every hour. After the 3rd run, how many copies of a file that arrived before the 1st run will be in the table?

A) 3 copies — one per run
B) 1 copy — COPY INTO is idempotent
C) 0 copies — only files after the 2nd run load
D) Depends on the COPY_OPTIONS setting

2. Which COPY INTO option allows you to re-ingest files that were already loaded?

A) FORMAT_OPTIONS ('reload' = 'true')
B) COPY_OPTIONS ('overwrite' = 'true')
C) COPY_OPTIONS ('force' = 'true')
D) SET force.reload = true

3. A team needs to ingest Kafka events stored as Avro files from S3. Which format should COPY INTO specify?

A) FILEFORMAT = PARQUET
B) FILEFORMAT = AVRO
C) FILEFORMAT = JSON
D) FILEFORMAT = KAFKA

4. What is the primary advantage of Parquet over CSV for analytical queries?

A) Parquet supports JSON nesting
B) Parquet is human-readable
C) Columnar layout enables column pruning and data skipping
D) Parquet has no compression overhead

5. A COPY INTO job fails midway. The engineer re-runs the job. What happens to partially loaded files?

A) Files are duplicated in the target table
B) Only newly arrived files are loaded; partial files are retried
C) COPY INTO tracks file-level state — already committed files are skipped, failed ones are retried
D) The entire job re-runs from scratch

6. Which statement about VALIDATE mode in COPY INTO is TRUE?

A) It writes data to the target table in a staging area
B) It performs a dry-run — checks schema and format but does not write data
C) It counts rows in source files only
D) VALIDATE is a separate SQL command, not a COPY INTO option

7. A CSV file has a header row and uses | as delimiter. Which FORMAT_OPTIONS correctly reads it?

A) 'header'='true', 'sep'='|'
B) 'header'='true', 'delimiter'='|'
C) 'hasHeader'='true', 'fieldSep'='|'
D) Both A and B are valid — 'sep' and 'delimiter' are aliases

8. Unity Catalog External Locations replace which older Databricks feature?

A) Delta sharing
B) DBFS mount points (dbutils.fs.mount)
C) Hive metastore tables

D) Cluster init scripts

9. A JSON file contains one JSON object per line (newline-delimited). Which FORMAT_OPTIONS is correct?

A) 'multiLine'='true'
B) 'multiLine'='false' (default)
C) 'jsonFormat'='ndjson'
D) No option needed — Spark auto-detects

10. Which file format stores its schema as a JSON document embedded in the file header, making it ideal for schema evolution in streaming pipelines?

A) Parquet
B) ORC
C) Avro
D) Delta

11. A data engineer wants to ingest customer records from a PostgreSQL database incrementally. Which approach is recommended?

A) Use COPY INTO to read from the database directly
B) Use JDBC with a cursor column (e.g., updated_at timestamp) to track loaded rows
C) Use Auto Loader for streaming database records
D) Use Lakeflow Connect to export CSV files first, then COPY INTO

12. Schema evolution in COPY INTO requires mergeSchema in two places. Which is correct?

A) FORMAT_OPTIONS only
B) COPY_OPTIONS only
C) Both FORMAT_OPTIONS AND COPY_OPTIONS
D) Neither — mergeSchema is automatic

13. A REST API endpoint returns 10,000 events per call. How should a data engineer ingest this incrementally?

A) Use COPY INTO with the API URL directly
B) Fetch via Python requests in a notebook, convert to DataFrame, write to Delta
C) Use Auto Loader to stream from the REST API
D) Use a scheduled job to call the API once per day

14. When should you use COPY_OPTIONS ('force' = 'true')?

A) Every time to ensure fresh data
B) After a schema change to reload historical data (use with caution)
C) To override file permissions
D) To enable multi-threading

15. Which cloud storage paths are valid for COPY INTO? (Select all that apply)

A) s3://bucket/path/
B) abfss://container@account.dfs.core.windows.net/
C) gs://bucket/path/
D) All of the above

Answer Key

Q1	**B**	COPY INTO tracks loaded files in the Delta transaction log. A file already recorded will be skipped on subsequent runs (unless force=true).
Q2	**C**	COPY_OPTIONS ('force' = 'true') disables the idempotency check and reloads all matching files.
Q3	**B**	Avro is the native format for Kafka event data. COPY INTO supports AVRO as a FILEFORMAT value.
Q4	**C**	Parquet's columnar format lets Spark read only queried columns (pruning) and skip row groups using min/max statistics.
Q5	**C**	COPY INTO uses Delta's transaction log for file-level tracking. Successfully committed files are not re-loaded; any uncommitted files are retried.
Q6	**B**	VALIDATE ALL or VALIDATE N ROWS performs a dry-run — useful to catch schema mismatches before the actual load.
Q7	**D**	Databricks/Spark accepts both 'sep' and 'delimiter' as synonyms. 'header'='true' is the correct header option.
Q8	**B**	External Locations in Unity Catalog replace legacy DBFS mount points, providing governed, credential-free access to cloud storage.
Q9	**B**	multiLine=false (the default) treats each line as a separate JSON record — correct for NDJSON/newline-delimited JSON files.
Q10	**C**	Avro stores its schema as a JSON document in the file header. This makes schema evolution straightforward in Kafka and streaming pipelines.
Q11	**B**	JDBC with cursor columns (updated_at, ID) enables incremental ingestion without CDC or complex setup.
Q12	**C**	mergeSchema must be set in BOTH FORMAT_OPTIONS (for source) AND COPY_OPTIONS (for target Delta table).
Q13	**B**	Fetch via Python requests, convert to DataFrame (using json.loads), and write to Delta. COPY INTO and Auto Loader do not support direct REST APIs.
Q14	**B**	force=true re-loads ALL files and can cause duplicates. Use only after schema changes or reprocessing needs.
Q15	**D**	All three cloud providers (AWS S3, Azure ADLS, GCS) are supported by COPY INTO.

Self-Assessment Checklist

- ■ I can write COPY INTO for CSV, JSON, Parquet, and Avro files
- ■ I understand idempotency and how COPY INTO tracks loaded files
- ■ I know when to use force=true and its risks
- ■ I can compare Parquet, Avro, ORC, and CSV trade-offs
- ■ I understand VALIDATE mode and when to use it
- ■ I know the cloud storage path formats for AWS, Azure, and GCS
- ■ I can explain External Locations and how they replace mount points
- ■ I understand the bronze → silver COPY INTO + MERGE pattern
- ■ I can write JDBC/ODBC code to ingest from databases
- ■ I understand cursor-based incremental loading
- ■ I can fetch and ingest from REST APIs
- ■ I understand schema enforcement vs. evolution
- ■ I know when mergeSchema is required in both FORMAT_OPTIONS and COPY_OPTIONS

■ **Exam Focus: Domain 2 (Data Ingestion & Loading) = 21% of 45 questions ≈ 9–10 questions. Master COPY INTO and Auto Loader (Week 4) thoroughly. These two topics cover ~60% of Domain 2.**

BONUS

BONUS: Advanced Patterns & Lab

Deep dive and hands-on

A. COPY INTO Edge Cases

A.1 Schema Mismatch Handling

When source files have columns not present in the target table, COPY INTO will fail unless mergeSchema is enabled:

■ **Schema merge on COPY INTO**

```
-- Source file has extra column "region" not in target
COPY INTO orders_bronze
FROM 's3://bucket/orders/'
FILEFORMAT = PARQUET
FORMAT_OPTIONS ('mergeSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
```

A.2 Directory Listing vs File Notification Modes

Mode	Mechanism	Scale	Latency	Used In
Directory Listing	Lists all files at trigger	Millions	Higher	COPY INTO (always)
File Notification	Cloud event subscription	Billions	Lower	Auto Loader (Week 4)

■ **COPY INTO always uses directory listing. Auto Loader (cloudFiles) can use either mode — file notification is recommended for high-volume streams.**

B. Hands-On Lab: End-to-End Ingestion

This lab simulates the Week 3 exam scenario: ingest CSV files from cloud storage into a Delta bronze table using COPY INTO, validate, then promote to silver.

B.1 Lab Setup: Create Bronze Table

■ **Lab: Create bronze table**

```
-- Create bronze table (schema-on-write)
CREATE TABLE IF NOT EXISTS catalog.bronze.raw_orders (
 order_id STRING,
 customer_id STRING,
 amount DOUBLE,
 order_date DATE,
 status STRING
) USING DELTA
LOCATION 'abfss://bronze@acct.dfs.core.windows.net/orders/';
```

B.2 Validate Before Load

■ **Lab: Validate step**

```
-- Validate before actual load
COPY INTO catalog.bronze.raw_orders
FROM 'abfss://raw@acct.dfs.core.windows.net/incoming/orders/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header'='true', 'inferSchema'='true')
```

```
VALIDATE ALL;

-- Review output for errors before proceeding
```

B.3 Load & Verify

■ Lab: Load and verify

```
-- Actual load
COPY INTO catalog.bronze.raw_orders
FROM 'abfss://raw@acct.dfs.core.windows.net/incoming/orders/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header'='true', 'inferSchema'='true');

-- Verify
SELECT COUNT(*) FROM catalog.bronze.raw_orders;
DESCRIBE HISTORY catalog.bronze.raw_orders LIMIT 3;
```

B.4 Silver Promotion with MERGE

■ Lab: Silver MERGE

```
MERGE INTO catalog.silver.orders AS tgt
USING catalog.bronze.raw_orders AS src
ON tgt.order_id = src.order_id
WHEN MATCHED AND src.status != tgt.status
 THEN UPDATE SET tgt.status = src.status,
 tgt.updated_at = CURRENT_TIMESTAMP()
WHEN NOT MATCHED
 THEN INSERT (order_id, customer_id, amount, order_date, status, updated_at)
 VALUES (src.order_id, src.customer_id, src.amount, src.order_date,
 src.status, CURRENT_TIMESTAMP());
```

REFERENCE **BONUS: Week 3 Master Reference** Quick lookup guide

C. COPY INTO Quick Reference

Keyword	Purpose	Example
FILEFORMAT	Source format	CSV | JSON | PARQUET | AVRO | ORC | TEXT
FORMAT_OPTIONS header	First row = column names (CSV)	'true'
FORMAT_OPTIONS delimiter	Field separator (CSV)	',', '|', '\t'
FORMAT_OPTIONS multiLine	Multi-line JSON objects	'true' (single-object) / 'false' (NDJSON)
FORMAT_OPTIONS mergeSchema	Allow new source columns	'true'
COPY_OPTIONS force	Re-load already loaded files	'true'
COPY_OPTIONS mergeSchema	Evolve Delta target schema	'true'
VALIDATE	Dry-run — no writes	VALIDATE ALL / VALIDATE 100 ROWS
FILES	Explicit file list	FILES = ('a.csv', 'b.csv')
PATTERN	Glob filter	PATTERN = '2024/*.parquet'

D. Key Exam Facts

- COPY INTO is idempotent — same files are not re-loaded (unless force=true)
- COPY INTO always uses directory listing (not file notification)
- Auto Loader (cloudFiles) is the streaming alternative to COPY INTO
- Parquet = columnar, embedded schema, best for analytics
- Avro = row-based, JSON schema in file header, best for Kafka/streaming
- ORC = columnar, Hive-native, older Hadoop workloads
- JSON = row-based, self-describing, best for APIs & semi-structured data
- CSV = universal, row-based, best for simple exports & Excel compat
- External Locations replace dbutils.fs.mount in Unity Catalog
- VALIDATE mode performs dry-run — no data is written
- mergeSchema must be set in BOTH FORMAT_OPTIONS and COPY_OPTIONS
- JDBC/ODBC enables incremental ingestion from databases with cursor columns
- REST APIs are fetched via Python requests, converted to DataFrame

E. Common Mistakes to Avoid

Mistake	What Happens	Correct Approach

Forgetting mergeSchema in FORMAT_OPTIONS or COPY_OPTIONS	Schema evolution fails silently or throws error	Set mergeSchema='true' in BOTH places
Using COPY INTO for streaming data	Only loads files once per run; doesn't handle real-time	Use Auto Loader for continuous streaming
multiLine=true for NDJSON (newline-delimited JSON)	Parser fails — expects one object, gets line-by-line	Use multiLine=false (default) for NDJSON
Using dbutils.fs.mount in new workloads	Legacy, no governance, credential exposure	Use External Locations with Unity Catalog
force=true every run to ensure fresh data	Duplicates if not handled downstream; performance cost	Use force=true only after schema changes

FRAMEWORK

F. Ingestion Method Decision Tree

Choose the right tool

Scenario	Best Method	Key Reason
One-time historical load from S3	COPY INTO	Batch, idempotent, SQL interface
Ongoing streaming from Kafka events	Auto Loader (cloudFiles)	Continuous, file notification, checkpointing
SaaS data (Salesforce, ServiceNow)	Lakeflow Connect	Managed connectors, no custom code
Custom REST API integration	JDBC / Python requests + notebooks	Lakeflow Connect not available
Daily CSV drop from partner	COPY INTO on schedule	Idempotent, handles any file count
High-volume IoT events (billions of files)	Auto Loader file notification	Directory listing fails at scale
Incremental database ingestion	JDBC with cursor column	Tracks state, no CDC needed

REVIEW

G. Domain 2 Comprehensive Review

Full ingestion strategy

Domain 2 Exam Coverage

Domain 2 (Data Ingestion & Loading) represents 21% of the exam (45 questions × 0.21 ≈ 9–10 questions). Week 3 covers COPY INTO, file formats, and idempotency. Week 4 covers Auto Loader and schema evolution. Together, these two weeks are essential for exam success.

Exam Patterns & Question Types

Pattern Type	Typical Question	What It Tests
Idempotency Scenarios	"After 5 runs of COPY INTO, how many copies of a file loaded on run 1?"	Understanding transaction log tracking
Format Selection	"Which format is best for streaming Kafka events?"	File format trade-offs (Avro, Parquet, JSON)
Option Syntax	"Which FORMAT_OPTIONS key enables schema evolution?"	Correct SQL syntax for COPY INTO
Failure Scenarios	"COPY INTO fails with schema mismatch. How to fix?"	Troubleshooting & mergeSchema usage
Integration Patterns	"Ingest customer records from PostgreSQL. Approach?"	JDBC/ODBC vs. Lakeflow Connect decision
Scale & Performance	"Ingesting billions of files. Use COPY INTO or Auto Loader?"	Understanding tool limitations (directory listing vs. file notification)

MOCK EXAM

H. Full-Length Mock Exam (20 Questions)

Timed practice

Instructions: Solve in 45 minutes (~2.25 min/question). Try without notes first. Check answers at the end.

1 An ETL pipeline uses COPY INTO to ingest CSV files from S3 every hour. Files arrive in /raw/orders/. Week 1: 100 files loaded. Week 2: 50 new files + 100 old files in directory. Week 2 COPY INTO run loads how many files?

A) 150 total
B) 50 new
C) 100 old
D) 0 (all cached)

2 Which statement about force=true is correct?

A) Always use force=true for safety
B) Re-loads ALL files; use only after schema changes
C) Required for incremental loading
D) Enables parallel COPY INTO

3 You ingest multi-line JSON (one object per file). FORMAT_OPTIONS should be:

A) 'multiLine'='true'
B) 'jsonFormat'='multiline'
C) 'mode'='multiline'
D) Default (no option)

4 Which cloud storage path is INVALID for COPY INTO?

A) s3://my-bucket/data/
B) gs://my-bucket/data/
C) hdfs://namenode/data/
D) abfss://container@account.dfs.core.windows.net/

5 Schema evolution with mergeSchema requires the setting in:

A) FORMAT_OPTIONS only
B) COPY_OPTIONS only
C) Both FORMAT_OPTIONS AND COPY_OPTIONS
D) Neither (automatic)

6 VALIDATE mode is used to:

A) Count rows in source
B) Perform dry-run (no write)
C) Enable parallel validation
D) Compress files

7 Which format stores schema as JSON in file header?

A) Parquet
B) Avro
C) ORC
D) JSON

8 For incremental JDBC ingestion from PostgreSQL, use:

A) Timestamp cursor column
B) CDC (Change Data Capture)
C) Full table scan each time
D) Auto Loader

9 REST API returns 100K events. Python fetch + write to Delta. Data type before write?

A) String
B) PySpark DataFrame
C) Bytes
D) Dict

10 External Locations replaces which legacy feature?

A) Delta Sharing
B) dbutils.fs.mount
C) Hive metastore
D) Cluster init scripts

11 Auto Loader file notification mode handles _____ of files per scale.

A) Millions
B) Billions
C) Thousands
D) Same as COPY INTO

12 COPY INTO result set includes:

A) Only num_affected_rows
B) num_inserted_rows + num_skipped_rows
C) Timestamp + partition
D) All of above

13 Parquet columnar advantage over CSV:

A) Human-readable
B) Column pruning + data skipping
C) No compression
D) Nesting support

14 ORC is best for:

A) Kafka streams
B) Hive-compatible workloads
C) REST APIs
D) Excel exports

15 Schema mismatch (source has extra column not in target):

A) Always fails
B) Silently ignores extra column
C) Fails unless mergeSchema='true'
D) Auto-creates column

16 JDBC connection secured with:

A) Secrets in notebook code
B) dbutils.secrets.get()
C) Environment variables
D) Hard-coded in query

17 Idempotency in COPY INTO guaranteed by:

A) File size check
B) CRC hash
C) Delta transaction log
D) Cluster cache

18 When to use Avro over Parquet?

A) Always for analytics
B) Kafka streaming (schema evolution)
C) Never
D) Only for compression

19 CSV options: header, delimiter, nullValue. All three are in:

A) FILEFORMAT
B) FORMAT_OPTIONS
C) COPY_OPTIONS
D) VALIDATE

20 MERGE INTO after COPY INTO enables:

A) Incremental ingestion
B) Schema evolution
C) SCD Type 2 / Dedup
D) Compression

Mock Exam Answer Key

Q1	**B**	COPY INTO tracks loaded files. Old 100 files are skipped; only 50 new ones load.
Q2	**B**	force=true re-loads ALL files. Use only after schema changes or reprocessing.
Q3	**A**	multiLine=true for single multi-line objects. Default false for NDJSON (one per line).
Q4	**C**	HDFS (hdfs://) is not supported. Only cloud storage (S3, GCS, ADLS) and DBFS.
Q5	**C**	mergeSchema must be in BOTH FORMAT_OPTIONS (source) AND COPY_OPTIONS (target).
Q6	**B**	VALIDATE ALL or VALIDATE N ROWS performs dry-run — checks schema, format, permissions; no write.
Q7	**B**	Avro embeds schema as JSON in file header, ideal for streaming & schema evolution.
Q8	**A**	Use timestamp (updated_at) or ID cursor column to track loaded rows. No CDC needed.
Q9	**B**	Fetch JSON via requests, parse with json.loads(), convert to DataFrame, write to Delta.
Q10	**B**	External Locations replace dbutils.fs.mount. Unity Catalog governs access.
Q11	**B**	Auto Loader file notification can handle billions of files; COPY INTO (directory listing) caps at millions.
Q12	**B**	COPY INTO returns num_affected_rows, num_inserted_rows, num_skipped_corrupt_files.
Q13	**B**	Parquet enables column pruning (read only needed columns) & data skipping (min/max stats).
Q14	**B**	ORC is columnar, Hive-native format. Used in older Hadoop/Hive workloads.
Q15	**C**	Schema mismatch fails unless mergeSchema='true' in BOTH FORMAT_OPTIONS and COPY_OPTIONS.
Q16	**B**	Use dbutils.secrets.get(scope, key) to retrieve credentials securely. Never hard-code.
Q17	**C**	Delta transaction log tracks each loaded file {'path': '...', 'loaded': true}.
Q18	**B**	Avro (row-based, schema in header) is ideal for Kafka streaming with evolving schema.
Q19	**B**	All CSV options (header, delimiter, nullValue) go in FORMAT_OPTIONS.
Q20	**C**	MERGE INTO enables SCD Type 2 (slowly changing dimensions) and deduplication after bronze load.

Scoring: 16–20 = Excellent | 13–15 = Good | 10–12 = Review domain | < 10 = Study COPY INTO & Auto Loader again

CHEAT SHEET

I. Final Cheat Sheet & Study Tips

Last-minute review

Essential SQL Patterns (Copy-Paste Ready)

■ **Week 3 Essential Patterns**

```
-- Pattern 1: Basic CSV batch load
COPY INTO bronze.sales FROM 's3://bucket/sales/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header'='true', 'delimiter'='|');

-- Pattern 2: Force reload (after schema change)
COPY INTO bronze.events FROM 's3://bucket/events/'
FILEFORMAT = PARQUET COPY_OPTIONS ('force'='true');

-- Pattern 3: Schema evolution
COPY INTO bronze.events FROM 's3://bucket/events/'
FILEFORMAT = JSON
FORMAT_OPTIONS ('multiLine'='false', 'mergeSchema'='true')
COPY_OPTIONS ('mergeSchema'='true');

-- Pattern 4: JDBC incremental
df = spark.read.format("jdbc")
 .option("url", "jdbc:postgresql://host:5432/db")
 .option("dbtable", "SELECT * FROM customers WHERE updated_at > '2024-01-01'")
 .load()
df.write.format("delta").mode("append").save("/mnt/bronze/customers")

-- Pattern 5: Bronze → Silver MERGE
MERGE INTO silver.orders tgt USING bronze.orders src
ON tgt.order_id = src.order_id
WHEN MATCHED THEN UPDATE SET * WHEN NOT MATCHED THEN INSERT *;
```

Last 24 Hours Before Exam

- ✓ Drill COPY INTO syntax (basic, CSV, JSON, force=true, VALIDATE, mergeSchema)
- ✓ Memorize file format trade-offs: Parquet (analytics), Avro (Kafka), CSV (simple), ORC (Hive)
- ✓ Practice idempotency scenarios: "What if job runs twice? How many copies?" (Answer: 1)
- ✓ Review External Locations vs mount points (External wins in UC workloads)
- ✓ Know cursor-based JDBC incremental pattern (timestamp/ID column)
- ✓ Understand mergeSchema MUST be in BOTH FORMAT_OPTIONS AND COPY_OPTIONS
- ✓ Skim official exam guide Section 2 (Data Ingestion) one more time
- ✓ Review Week 4 Auto Loader preview (cloudFiles, checkpointing)
- ✓ Take mock exam (20 Qs) and review wrong answers
- ✓ REST; DO NOT cram new material last night

Domain 2 Exam Tips

■ Allocate ~9–10 questions per domain (45 Qs ÷ 6 domains with Domain 2 at 21%). Manage time!

■ If question asks "Best tool for X", use the decision tree: file scale? CDC? SaaS? → COPY INTO | Auto Loader | Lakeflow Connect

■ Format questions: Always pick Parquet for lakehouse analytics, Avro for Kafka. JSON for APIs.

■■ Watch for "mergeSchema" trap: Both FORMAT_OPTIONS AND COPY_OPTIONS required. Not just one.

■ Idempotency questions: COPY INTO idempotent by default (delta log). Only re-loads new files.

■ Cloud path format questions: s3://, gs://, abfss://. HDFS not supported.

■ Troubleshooting: Schema mismatch → mergeSchema. Scale → Auto Loader. Streaming → Auto Loader. OneTime → COPY INTO.

■ When unsure, re-read question for keywords: "incremental", "streaming", "historical", "scale", "schema evolution"

LAB

J. Complete Lab Scenarios

Real-world end-to-end workflows

Lab 1: E-Commerce Orders — CSV Ingestion + Dedup

Scenario: Daily CSV order file from partner system. Multiple runs possible; need to deduplicate and track incremental changes.

■ Lab 1: CSV Dedup Pattern

```
-- Step 1: Create bronze table
CREATE TABLE IF NOT EXISTS catalog.bronze.orders_raw (
 order_id STRING,
 customer_id STRING,
 total_amount DOUBLE,
 order_date DATE,
 _ingestion_ts TIMESTAMP DEFAULT CURRENT_TIMESTAMP()
) USING DELTA;

-- Step 2: Incremental COPY INTO (idempotent)
COPY INTO catalog.bronze.orders_raw
FROM 's3://partner-bucket/daily-exports/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header'='true', 'inferSchema'='false')
PATTERN = '*.csv'; -- Only CSV files

-- Step 3: Promote to silver with dedup
CREATE OR REPLACE TABLE catalog.silver.orders AS
SELECT * FROM (
 SELECT *,
   ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY _ingestion_ts DESC) AS rn
 FROM catalog.bronze.orders_raw
) WHERE rn = 1;

-- Step 4: Validate
SELECT COUNT(DISTINCT order_id) as unique_orders FROM catalog.silver.orders;
```

Lab 2: Database Incremental Ingestion — JDBC Cursor

Scenario: Ingest customer changes from PostgreSQL daily. Only load records updated since last run using timestamp column.

■ Lab 2: JDBC Incremental Cursor Pattern

```
# Step 1: Get last load timestamp
from datetime import datetime, timedelta

last_load = spark.read.parquet('/mnt/bronze/customers') \
 .agg({'updated_at': 'max'}) \
 .collect()[0][0]
last_load_ts = last_load or datetime(2024, 1, 1)

# Step 2: JDBC read with cursor filter
df = spark.read \
 .format('jdbc') \
 .option('url', 'jdbc:postgresql://prod-db.internal:5432/main') \
```

```
  .option('dbtable', f"(SELECT * FROM customers WHERE updated_at > '{last_load_ts}') AS q") \
  .option('user', dbutils.secrets.get('db', 'user')) \
  .option('password', dbutils.secrets.get('db', 'pass')) \
  .option('numPartitions', 4) \
  .load()

print(f'Loaded {df.count()} new/updated records')

# Step 3: Write to bronze (append only)
df.write.format('delta').mode('append').save('/mnt/bronze/customers')
```

Lab 3: Streaming API Webhook → Delta Lake

Scenario: REST API webhook sends events. Parse JSON, transform, load to Delta in near-real-time.

■ Lab 3: REST API Webhook Pattern

```
import requests, json, time
from pyspark.sql import Row
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, TimestampType

# Step 1: Fetch from webhook API
url = 'https://api.internal.com/v2/events'
headers = {'Authorization': f"Bearer {dbutils.secrets.get('api', 'token')}"}

events = []
for page in range(1, 6):  # Paginated API
    resp = requests.get(f'{url}?page={page}', headers=headers)
    data = resp.json()
    events.extend(data['events'])
    if not data.get('has_more'): break

# Step 2: Transform to DataFrame
schema = StructType([
    StructField('event_id', StringType()),
    StructField('user_id', StringType()),
    StructField('event_type', StringType()),
    StructField('value', DoubleType()),
    StructField('timestamp', TimestampType())
])

df = spark.createDataFrame(events, schema=schema)

# Step 3: Write to Delta
df.write.format('delta').mode('append').save('/mnt/bronze/events')
```

TROUBLESHOOT

K. Troubleshooting Deep Dive

Common errors & solutions

Schema Mismatch Error
Symptom: COPY INTO fails: 'Unable to infer schema'

Solution: Set FORMAT_OPTIONS ('mergeSchema'='true') AND COPY_OPTIONS ('mergeSchema'='true')

File Not Found
Symptom: COPY INTO fails: 's3://bucket/path/ does not exist'

Solution: Check path spelling, bucket policy, and cloud credentials. Verify External Location permissions.

Permission Denied on S3
Symptom: COPY INTO fails: 'Access Denied'

Solution: Verify IAM role has s3:GetObject, s3:ListBucket. Check Storage Credential in Unity Catalog.

Duplicate Rows After Rerun
Symptom: COPY INTO run 2 has duplicates from run 1

Solution: COPY INTO should be idempotent by default. Check if force=true was used unintentionally.

CSV Header Not Detected
Symptom: Column names = 'col1', 'col2' instead of actual headers

Solution: Set FORMAT_OPTIONS ('header'='true')

JSON Parsing Error
Symptom: COPY INTO fails: 'Invalid JSON'

Solution: Check multiLine setting: 'false' for NDJSON, 'true' for multi-line JSON objects.

JDBC Timeout
Symptom: JDBC read hangs or times out

Solution: Increase numPartitions, add network ACL for Databricks to DB, reduce query size (use cursor column).

Memory Issues During Load
Symptom: Cluster OOM during large COPY INTO

Solution: Reduce parallelism, use smaller PATTERN, partition source by date.

REFERENCE

L. Complete SQL Reference

All COPY INTO options

Full COPY INTO Syntax

■ **Full COPY INTO Syntax**

```
COPY INTO [catalog.][schema.]table_name
FROM 's3://bucket/path/' | 'gs://bucket/' | 'abfss://container@account.dfs.core.windows.net/'
FILEFORMAT = CSV | JSON | PARQUET | AVRO | ORC | TEXT
FORMAT_OPTIONS (
  'key1' = 'value1',
  'key2' = 'value2'
)
COPY_OPTIONS (
  'key1' = 'value1'
)
[FILES = ('file1.csv', 'file2.parquet', ...)]
[PATTERN = 'glob_pattern']
[VALIDATE ALL | VALIDATE 100 ROWS];
```

FORMAT_OPTIONS by File Type

CSV: header='true', delimiter=',', quote='"', escape='\\', nullValue='NULL', inferSchema='true'

JSON: multiLine='false' (default for NDJSON), mergeSchema='true' (for evolution)

Parquet: mergeSchema='true' (for schema evolution)

Avro: No special options (schema in file header)

ORC: No special options

COPY_OPTIONS Reference

force='true': Re-load ALL files (use sparingly)

mergeSchema='true': Evolve Delta table schema (must pair with FORMAT_OPTIONS)

Pro Tip: Always test with VALIDATE ALL first. Schema issues caught early save time!

PERF

M. Performance Tuning Tips

Optimize COPY INTO & JDBC

COPY INTO Performance

- Use PATTERN to narrow file scope: PATTERN = '2024-01/*.parquet' (month partition)
- Set mergeSchema='true' only if schema truly evolves. Default (false) is faster.
- Avoid force=true in production. Let idempotency handle retries.
- For billions of files → Auto Loader file notification, not COPY INTO directory listing

JDBC Performance

- Use cursor column (timestamp, ID) to filter rows. Avoid full table scan.
- Set numPartitions to 4–8 to parallelize reads across DB connections.
- Use lowerBound/upperBound to further split work: .option('lowerBound', 1000) .option('upperBound', 1000000)
- Increase fetchsize for large result sets: .option('fetchsize', '10000')

File Format Best Practices

- Parquet: Default for lakehouse. Columnar compression saves 70–90% storage vs CSV.
- Avro: Use only for streaming/Kafka. Row-based; slower for analytics than Parquet.
- ORC: Legacy Hive workloads. Not recommended for new projects.
- CSV: Universal compatibility but slow for analytics. Use only for exports or legacy sources.

Flash Cards: Key Concepts

Use these flash cards for quick memory drills. Cover the answer (right side) and test yourself.

Q: What is COPY INTO idempotency?

A: COPY INTO tracks files in Delta log. Same file won't load twice unless force=true. Safe for retries.

Q: When do you use force=true?

A: After schema changes or reprocessing needs. Use with caution — re-loads ALL files, risk of duplicates.

Q: FILEFORMAT for Kafka events?

A: AVRO. Row-based, schema in header (JSON), perfect for schema evolution in streaming.

Q: Best format for lakehouse analytics?

A: PARQUET (Delta). Columnar, embedded schema, enables column pruning & data skipping.

Q: multiLine='true' or 'false' for NDJSON?

A: 'false' (default). NDJSON = one JSON per line. 'true' only for multi-line JSON objects.

Q: mergeSchema in BOTH places — why?

A: FORMAT_OPTIONS (source), COPY_OPTIONS (target). Missing either breaks schema evolution.

Q: COPY INTO for streaming data?

A: NO. COPY INTO is batch only. Use Auto Loader (cloudFiles) for continuous streaming.

Q: External Location vs mount point?

A: External Locations (UC) replace mount points. Better: governed, credential-free, scalable.

Q: How many copies after 3 COPY INTO runs?

A: 1 copy. Idempotent by default. File tracked in log after 1st load.

Q: JDBC for incremental database ingestion — how?

A: Use cursor column (timestamp/ID). Filter with WHERE clause: 'updated_at > LAST_LOAD_TS'.

Q: REST API with 100K events — approach?

A: Fetch via Python requests, parse JSON, create DataFrame, write to Delta.

Q: What does VALIDATE ALL do?

A: Dry-run. Checks schema, format, permissions — NO data written. Safe pre-flight check.

Q: directory listing vs file notification scale?

A: Directory listing (COPY INTO): millions. File notification (Auto Loader): billions.

Q: ORC best for — what workload?

A: Hive-compatible workloads. Columnar, Hadoop-native. Not recommended for new projects.

Q: CSV FORMAT_OPTIONS: header, delimiter, nullValue?

A: header='true', delimiter=',', nullValue='NULL'. Set inferSchema='true' to auto-detect types.

Q: Schema evolution in Auto Loader?

A: cloudFiles.schemaEvolutionMode='addNewColumns'. Detects new columns per batch.

Q: mergeSchema='true' — what happens to existing rows?

A: New columns get NULL for existing rows. Schema grows, data intact.

Q: JDBC connection — secure credentials?

A: Use dbutils.secrets.get(scope, key). NEVER hard-code in query.

Q: MERGE INTO after COPY INTO — why?

A: SCD Type 2 (track changes), deduplication, upsert (match + update/insert).

Q: Cloud paths COPY INTO supports?

A: s3://, gs://, abfss://. NOT hdfs://, dbfs:/, or file:// paths.

Quick Reference: COPY INTO vs Alternatives

Aspect	COPY INTO	Auto Loader	Lakeflow Connect	JDBC/REST
Interface	SQL	PySpark	Managed UI	Python/SQL
Idempotent	✓ (default)	✓ (checkpoint)	✓ (state tracking)	Manual (cursor)
Scale	Millions	Billions	Enterprise	On-demand
Schema Inference	Single pass	Incremental	Target-driven	Implicit
Source Types	Cloud files	Cloud files	Databases, SaaS	Databases, APIs
Recommended For	Batch ETL	Streaming	Enterprise data	Custom sources

Last-Minute Exam Cram

✓ **COPY INTO Syntax:** COPY INTO table FROM 's3://bucket/' FILEFORMAT = FORMAT FORMAT_OPTIONS (...) COPY_OPTIONS (...)

✓ **idempotency guarantee:** Delta log tracks files. Same file not re-loaded (unless force=true).

✓ **Schema evolution:** mergeSchema='true' in BOTH FORMAT_OPTIONS AND COPY_OPTIONS

✓ **File formats ranked:** Parquet (analytics) > Avro (streaming) > ORC (Hive) > JSON (APIs) > CSV (simple)

✓ **multiLine default:** false (NDJSON per default). true only for single multi-line JSON objects.

✓ **External Location:** Replaces mount points. Unity Catalog governed. Credential-free.

✓ **JDBC incremental:** Use cursor column (timestamp/ID). SELECT * WHERE updated_at > LAST_LOAD.

✓ **REST API pattern:** requests.get() → json.loads() → spark.createDataFrame() → write.format('delta')

✓ **Scale limits:** COPY INTO: millions (directory listing). Auto Loader: billions (file notification).

✓ **When to use force=true:** After schema changes only. Causes re-loads — duplicates if not handled.